

4.6 几何解释：投影与误差最小化

前一节将线性回归写成矩阵形式。设训练集包含 n 个样本， X 是由所有样本特征组成的设计矩阵， y 是由对应真实标签组成的向量， θ 是包含截距和各特征系数的参数向量。模型在全部样本上的预测向量为

$$\hat{y} = X\theta.$$

其中 $\hat{y}$ 表示预测值组成的向量。从代数角度看，最小二乘法通过最小化 $\|y - X\theta\|_2^2$ 求得参数 θ ， $\|\cdot\|_2$ 表示欧氏范数。从几何角度看，这一过程并不是任意地调整参数，而是在模型能够表示的范围内，寻找距离真实标签最近的预测向量。这个解释有助于理解平方误差、正规方程和残差之间的关系。

先从几何图像出发

矩阵 X 的每一列都可以看作一个特征方向。例如，若第一列是全 1 向量，它对应截距方向；其余列分别对应不同输入特征的变化方向。这些列向量的所有线性组合构成一个子空间，称为 X 的**列空间** (column space)，记为 $C(X)$ 。由于任意预测向量都具有 $\hat{y} = X\theta$ 的形式，因此

$$\hat{y} \in C(X).$$

这意味着模型不能生成任意的 n 维向量，只能生成列空间中的向量；这里的维数 n 正是训练样本数。真实标签向量 y 未必恰好落在 $C(X)$ 中，因此通常不能找到参数使 $X\theta = y$ 完全成立。最小二乘法所做的事情是：在 $C(X)$ 中选择一个向量 $\hat{y}$ ，使它与 y 的欧氏距离最小。图 1 给出了这一过程的二维示意。

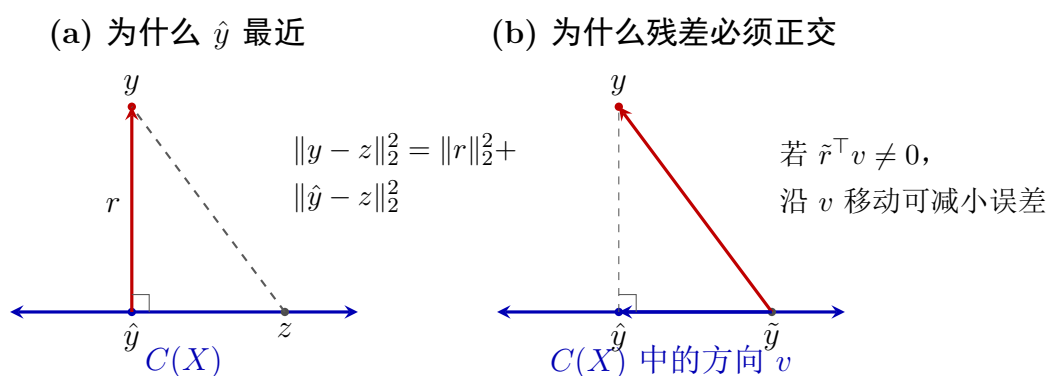


图 1: 最小二乘法的投影解释。左图中， z 表示列空间中的任意其他候选预测，投影点 $\hat{y}$ 比 z 更接近 y ；右图中， $\tilde{y}$ 表示一个尚未达到最优的候选预测， $\tilde{r} = y - \tilde{y}$ 是它的残差。若 $\tilde{r}$ 在某个可表示方向 v 上仍有分量，就还可以沿该方向移动并减小误差。

图 1(a) 中， θ^* 表示使平方误差达到最小的最优参数，因而 $\hat{y} = X\theta^*$ 是对应的最优预测向量，称为 y 在 $C(X)$ 上的**正交投影**。从 $\hat{y}$ 指向 y 的向量 $r = y - \hat{y}$ 称为残差向

量。这里“正交”的含义是：残差 r 垂直于列空间中的每一个方向。图 1(b) 说明了原因：取列空间中的任意非零方向向量 v ，若残差在该方向上仍有分量，即内积 $r^\top v \neq 0$ ，那么沿 v 或 $-v$ 的方向稍微移动预测向量，就可以进一步靠近 y ；此时 $\hat{y}$ 就不可能是最近点。其中上标 $\top$ 表示向量转置， $r^\top v$ 是 r 与 v 的内积。

为什么投影点一定最近

下面说明正交投影为何能够最小化平方误差。设 z 是列空间 $C(X)$ 中任意另一个候选预测向量。由于 $\hat{y}$ 和 z 都属于 $C(X)$ ，所以 $\hat{y} - z$ 也属于 $C(X)$ 。又因为残差 r 与整个列空间正交，所以

$$r \perp (\hat{y} - z).$$

将 $y - z$ 分解为两部分，可得

$$y - z = (y - \hat{y}) + (\hat{y} - z) = r + (\hat{y} - z).$$

这两个分量彼此正交，因此由勾股关系有

$$\|y - z\|_2^2 = \|r\|_2^2 + \|\hat{y} - z\|_2^2 \geq \|r\|_2^2. \quad (1)$$

而 $\|r\|_2^2 = \|y - \hat{y}\|_2^2$ 。式(1)表明，对列空间中任意其他向量 z ，它与 y 的距离平方都不小于投影点 $\hat{y}$ 与 y 的距离平方。因此， $\hat{y}$ 就是最小二乘意义下的最优预测向量。

这个结论也解释了平方误差的含义：

$$\|y - X\theta\|_2^2$$

并不只是一个便于计算的损失函数，它正是标签向量 y 到模型可表示空间 $C(X)$ 的距离平方。最小二乘法选择的参数 θ^* ，就是使这一距离最短的参数。

残差正交如何导出正规方程

还可以从最小化条件直接说明残差为何正交。取列空间中的任意非零方向向量 $v \in C(X)$ ，再用实数 t 表示沿该方向移动的步长。于是， $\hat{y} + tv$ 表示从 $\hat{y}$ 出发得到的新预测向量： $t > 0$ 表示沿 v 移动， $t < 0$ 表示沿 $-v$ 移动， $t = 0$ 则保持在原来的 $\hat{y}$ 。定义

$$g(t) = \|y - (\hat{y} + tv)\|_2^2.$$

这里 $g(t)$ 是关于步长 t 的一元函数，用来记录预测向量沿方向 v 移动后，平方误差怎样变化。将 $r = y - \hat{y}$ 代入并展开，可得

$$g(t) = \|r - tv\|_2^2 = \|r\|_2^2 - 2tr^\top v + t^2\|v\|_2^2.$$

若某个方向 $v \in C(X)$ 满足 $r^\top v \neq 0$ ，取步长

$$t = \frac{r^\top v}{\|v\|_2^2},$$

就有 $g(t) = \|r\|_2^2 - (r^\top v)^2 / \|v\|_2^2 < g(0)$, 这与 $\hat{y}$ 已经最优矛盾。因此, 对任意 $v \in C(X)$ 必有 $r^\top v = 0$, 即 r 与列空间正交。

列空间由 X 的各列张成。于是, 残差与每一列都正交可以简洁地写成

$$X^\top r = 0.$$

代入 $r = y - X\theta^*$, 得到

$$X^\top (y - X\theta^*) = 0,$$

即

$$X^\top X\theta^* = X^\top y.$$

这正是第 4.5 节中通过求导得到的正规方程。因而, 正规方程既可以看作平方误差的一阶最优条件, 也可以看作残差与模型列空间正交的几何条件。

4.7 概率解释：高斯噪声与最大似然估计

第 4.6 节从几何角度说明了最小二乘法：在线性模型能够表示的列空间中, 寻找距离标签向量最近的预测。还可以从另一个问题出发理解线性回归：如果数据是由某种带随机性的机制产生的, 那么什么样的参数最能解释已经观察到的数据？这一问题引出概率模型和最大似然估计。

从预测函数到数据生成机制

在线性回归中, 模型给定输入 x_i 后输出预测值 $w^\top x_i + b$ 。概率解释进一步认为, 真实标签并不会严格等于这个预测值, 而是在它附近随机波动。换言之, 模型不仅描述一个预测函数, 还描述了在给定输入和参数时, 标签 y_i 出现的条件概率密度:

$$p(y_i | x_i; w, b).$$

参数估计所要回答的问题是：已经观测到一组样本 (x_i, y_i) 后, 应该选择哪一组 w, b , 才能使这些标签在模型下显得最合理？把参数固定后, 观测数据出现的可能性是概率；把观测数据固定后, 考察不同参数使这些数据出现的可能性, 就得到似然 (likelihood)。

若样本在给定参数后相互独立, 则整个训练集的似然为各个样本条件概率的乘积:

$$L(w, b) = \prod_{i=1}^n p(y_i | x_i; w, b).$$

极大似然估计 (maximum likelihood estimation, MLE) 的思想是选择使似然最大的参数:

$$(w^*, b^*) = \arg \max_{w, b} L(w, b).$$

它表达的并不是“寻找最常见的标签”, 而是寻找一组参数, 使整个已观测数据集在该模型下最容易出现。

为什么通常使用对数似然

直接处理似然时会遇到两个问题：其一，许多小于 1 的概率相乘，数值上容易变得极小；其二，乘积不利于推导和优化。由于对数函数单调递增，最大化似然与最大化对数似然得到的参数完全相同。对数将乘积变为求和：

$$\ell(w, b) = \log L(w, b) = \sum_{i=1}^n \log p(y_i | x_i; w, b).$$

因此，许多机器学习训练过程也写成最小化负对数似然：

$$\min_{w, b} -\ell(w, b).$$

这一步建立了概率模型与损失函数之间的联系。损失函数不再只是人为指定的一种误差度量，而可以被看作“数据在当前参数下不合理程度”的数值表达。不同的数据生成假设会导出不同的负对数似然，也就会导出不同的训练损失。

高斯噪声下的线性回归

对于连续型预测任务，一个常见假设是：标签由线性部分和均值为零的随机噪声共同组成，

$$y_i = w^\top x_i + b + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2). \quad (2)$$

其中 σ^2 描述噪声的波动强度。由式(2) 可知，在给定 x_i 时， y_i 的条件分布为

$$y_i | x_i \sim \mathcal{N}(w^\top x_i + b, \sigma^2).$$

也就是说，模型预测值 $w^\top x_i + b$ 是条件分布的中心，真实标签会围绕这个中心上下波动。预测误差越大，观测到该标签的概率密度越低。

在样本条件独立的假设下，训练集的似然可以写成

$$L(w, b) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - w^\top x_i - b)^2}{2\sigma^2}\right).$$

对上式取对数，得到

$$\ell(w, b) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^\top x_i - b)^2.$$

当 σ^2 固定时，第一项与 w, b 无关，第二项前的系数为负常数。因此，最大化对数似然等价于最小化平方误差：

$$\arg \max_{w, b} \ell(w, b) = \arg \min_{w, b} \sum_{i=1}^n (y_i - w^\top x_i - b)^2.$$

这就给出了最小二乘法的概率解释：若误差服从独立同分布的高斯分布，则最小二乘解恰好是最大似然解。

损失函数反映了什么假设

平方误差并非对所有数据都天然正确。它隐含了三个重要前提：误差以零为中心、正负偏差同等对待，并且较大偏差出现的概率会按高斯形式快速下降。在这些前提合理时，平方误差既有清晰的统计含义，又便于求导和优化。

如果噪声机制不同，合适的损失函数也可能不同。例如，含有大量极端误差时，平方误差会对这些样本赋予很大权重，模型可能变得不稳定；若标签是二分类变量，则高斯噪声假设不再适用，通常需要从伯努利分布出发，并得到后续章节中的交叉熵损失。因而，选择损失函数的关键不只是计算方便，更在于它是否与任务的输出形式和数据生成机制相匹配。

常见任务中，概率假设与训练损失之间的对应关系可以概括为表 1。这里的要点不是机械记忆公式，而是先判断输出变量的类型和噪声特点，再选择与之匹配的概率模型和损失函数。

表 1:数据生成假设与常见训练损失的对应关系

输出或噪声特点	概率模型	由负对数似然得到的典型损失
连续标签，误差近似高斯	高斯分布 $\mathcal{N}(\mu, \sigma^2)$	平方误差；较大偏差受到更强惩罚
二分类标签 $y \in \{0, 1\}$	伯努利分布	二元交叉熵；衡量预测概率与真实标签的差异
连续标签，可能有较多极端误差	拉普拉斯分布	绝对误差；相比平方误差对异常值更稳健

4.8 梯度下降求解线性回归

前两节分别从几何和概率角度解释了最小二乘法，但这些解释仍未回答一个实际问题：当参数很多，或者模型没有可以直接计算的解析解时，应当怎样找到使损失较小的参数？正规方程通过一次矩阵求解得到线性回归的最优参数，梯度下降（gradient descent）则采用另一条路线：从一组初始参数出发，观察损失在当前位置增长最快的方向，再向相反方向移动一小步。不断重复这一过程，参数便逐渐靠近损失函数的最低点。

线性回归本身并不依赖梯度下降。这里选择线性回归作为例子，是因为它既有正规方程给出的最优解，又有结构清楚的损失曲面，因而可以直接观察迭代结果是否正确。更复杂模型虽然可能没有解析解，但其训练过程仍然遵循相同的“计算损失、计算梯度、更新参数”框架。

参数空间中的损失曲面

考虑只有一个输入变量的线性模型

$$\hat{y} = \theta_0 + \theta_1 x.$$

其中， θ_0 和 θ_1 分别表示截距和斜率。给定 n 个观测样本 (x_i, y_i) ，均方误差为

$$J(\theta_0, \theta_1) = \frac{1}{n} \sum_{i=1}^n (\theta_0 + \theta_1 x_i - y_i)^2 \quad (3)$$

每一组参数 (θ_0, θ_1) 都对应一个损失值。以两个参数为水平坐标、损失为竖直坐标，即得到图 2 所示的损失曲面。

实验数据的总体关系接近 $y = 1.2 + 1.7x$ ，并在标签中加入固定扰动。由于输入 x 未中心化，截距与斜率相互耦合，均方误差形成倾斜、狭长的凸谷。图中的参数轨迹先沿陡峭方向快速降低损失，进入谷地后再沿较平缓的方向逐步接近最小二乘解。

为什么沿负梯度方向移动

设当前参数向量为 θ ，其中包含模型的所有待估计参数；在本例中， $\theta = (\theta_0, \theta_1)^\top$ 。若参数发生一个较小变化 Δ ，可微损失函数在当前点附近的一阶泰勒近似为

$$J(\theta + \Delta) \approx J(\theta) + \nabla_\theta J(\theta)^\top \Delta. \quad (4)$$

这里， $\nabla_\theta J(\theta)$ 是损失函数关于各参数偏导数组成的梯度向量，它指出损失在当前位置增长最快的方向； Δ 表示本次参数移动的方向和距离。

选择

$$\Delta = -\eta \nabla_\theta J(\theta),$$

其中 $\eta > 0$ 称为学习率，用来控制一步移动的大小。将其代入式(4)，得到

$$\nabla_\theta J(\theta)^\top \Delta = -\eta \|\nabla_\theta J(\theta)\|_2^2 \leq 0.$$

因此，只要学习率使这一步仍处于局部近似有效的范围内，沿负梯度方向移动就会使损失降低。相应的参数更新规则为

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta J(\theta^{(t)}), \quad (5)$$

其中，上标 t 表示当前迭代轮次， $\theta^{(t)}$ 是第 t 轮更新前的参数， $\theta^{(t+1)}$ 是更新后的参数。普通梯度下降只使用当前位置的梯度，不包含动量、惯性、随机重启或模拟退火等机制。

学习率不能脱离损失曲面的形状来理解。 η 太小时，每一步都很短，收敛速度较慢； η 太大时，一阶近似可能不足以描述走出较远距离后的函数变化，参数可能跨过谷底来回震荡，甚至使损失不断增大。图 2 使用 $\eta = 0.04$ ，轨迹在前几轮迅速进入谷地，之后较平稳地沿谷底移动。

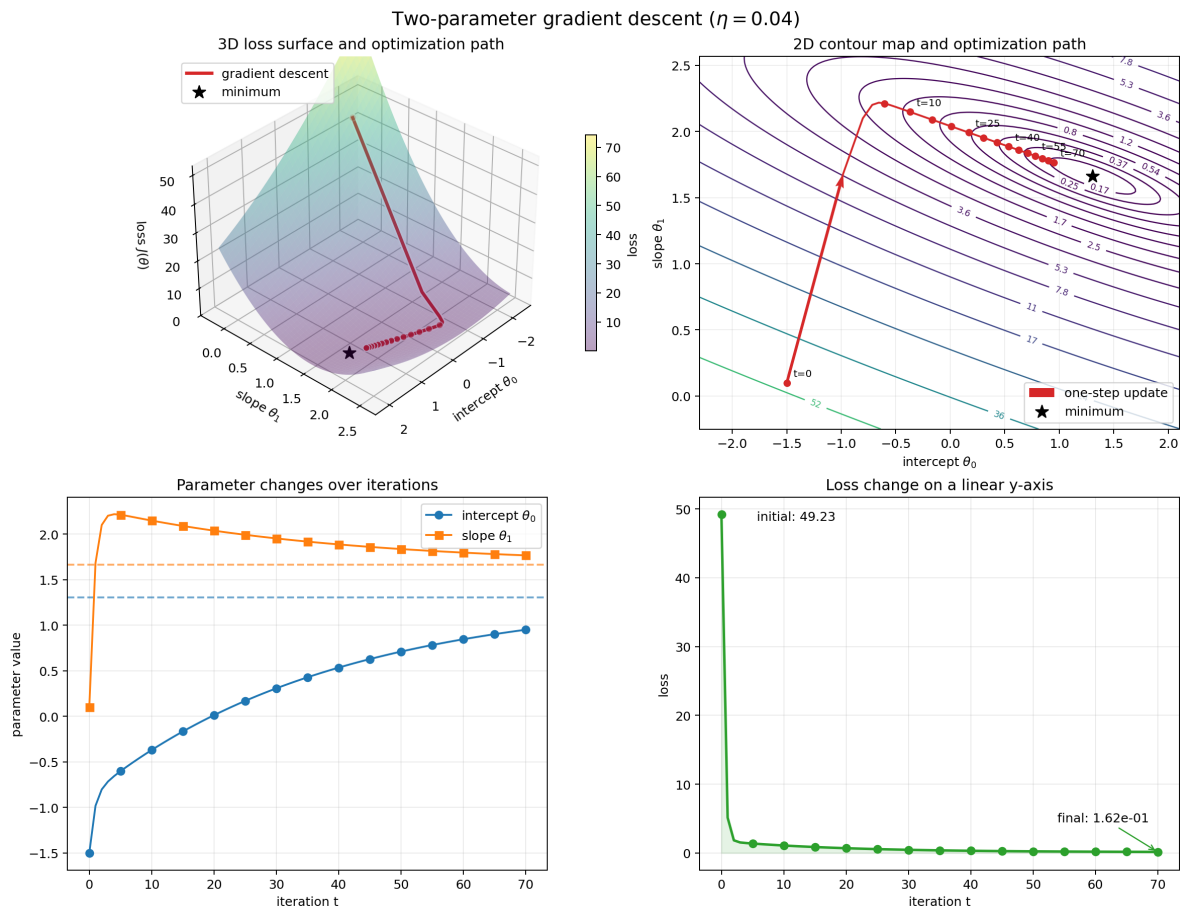


图 2: 双参数线性回归的梯度下降过程。左上图给出均方误差曲面及参数轨迹; 右上图是同一曲面的等高线表示, 红点每隔 5 轮记录一次, 箭头表示记录位置处的一步更新方向, 黑色五角星表示最小二乘解; 左下图记录截距和斜率的变化; 右下图记录普通线性纵轴下的损失变化。

线性回归中的梯度

将所有样本写成矩阵形式，设计矩阵 X 的每一行对应一个样本，第一列为全 1 向量，用来表示截距； y 是真实标签向量。预测向量为 $X\theta$ ，均方误差写为

$$J(\theta) = \frac{1}{n} \|X\theta - y\|_2^2. \quad (6)$$

令 $e = X\theta - y$ 表示“预测值减真实值”得到的误差向量，则式(6) 为 $J(\theta) = e^\top e/n$ 。对参数向量求导可得

$$\nabla_\theta J(\theta) = \frac{2}{n} X^\top (X\theta - y). \quad (7)$$

这个式子可以按“误差如何反馈到参数”来理解。向量 $X\theta - y$ 给出每个样本当前预测得过高还是过低，左乘 $X^\top$ 则把这些样本误差按照各特征的取值汇总到对应参数上。最后乘以 $2/n$ ，得到平均平方误差对各参数的变化率。

把式(7) 代入式(5)，线性回归的批量梯度下降更新为

$$\theta^{(t+1)} = \theta^{(t)} - \eta \frac{2}{n} X^\top (X\theta^{(t)} - y).$$

这里“批量”表示每次更新都使用全部 n 个训练样本计算梯度。由于同一轮中所有参数都由同一个旧参数向量 $\theta^{(t)}$ 计算得到，因此它们应当同时更新。

算法步骤与停止条件

线性回归的批量梯度下降可以归纳为以下过程：

1. 选择初始参数 $\theta^{(0)}$ 、学习率 η 和最大迭代次数。
2. 在第 t 轮计算预测向量 $\hat{y}^{(t)} = X\theta^{(t)}$ 。
3. 计算当前损失 $J(\theta^{(t)})$ 和梯度 $2X^\top (X\theta^{(t)} - y)/n$ 。
4. 按照式(5) 同时更新所有参数。
5. 当达到最大迭代次数，或损失、参数和梯度的变化已经足够小时停止。

最大迭代次数是最简单的停止条件，但它不能判断模型是否已经收敛。实际训练中还可以观察相邻两轮的损失差 $|J(\theta^{(t+1)}) - J(\theta^{(t)})|$ 、参数变化量 $\|\theta^{(t+1)} - \theta^{(t)}\|_2$ 或梯度范数 $\|\nabla J(\theta^{(t)})\|_2$ 。这些量足够小时，说明继续迭代带来的变化已经很小。不过，损失下降缓慢也可能是学习率过小或曲面过于狭长，不能仅凭一条平缓曲线断定已经达到最优解。

Python 实操：观察参数、梯度与损失

本节实验将参数初始化为 $(-1.5, 0.1)^\top$ ，使用学习率 0.04 进行 70 轮批量梯度下降。核心算法只需要保留损失、梯度和一次参数更新：

```
for _ in range(steps + 1):
    residuals = X @ theta - y
    loss = np.mean(residuals ** 2)
    gradient = 2 / len(y) * X.T @ residuals
    theta = theta - learning_rate * gradient
```

为了避免输出过密，程序仍然逐轮更新参数，但只每隔 5 轮记录一次结果。表 2 选取其中若干轮次，展示参数、梯度和损失的共同变化。

表 2: 梯度下降实验的部分迭代记录

轮次 t	θ_0	θ_1	$\partial J / \partial \theta_0$	$\partial J / \partial \theta_1$	$J(\theta)$
0	-1.5000	0.1000	-12.9575	-39.5768	49.2305
5	-0.6029	2.2123	-1.2356	0.2950	1.3771
10	-0.3704	2.1484	-1.0707	0.3098	1.0896
20	0.0118	2.0378	-0.8262	0.2391	0.6968
40	0.5342	1.8866	-0.4919	0.1424	0.3237
70	0.9498	1.7663	-0.2260	0.0654	0.1620

第 0 轮时两个梯度分量的绝对值较大，因此第一次更新带来的参数变化和损失下降最明显。进入谷地后，斜率 θ_1 先超过最小二乘解附近的取值，再逐渐减小，而截距 θ_0 持续增大。单个参数未必单调接近最优值，判断优化过程应结合总体损失、梯度和完整参数轨迹。

右下角使用普通线性纵轴。损失从 49.2305 快速降到第 5 轮的 1.3771，随后下降变慢：初始阶段消除了陡峭方向上的大误差，后续则沿狭长谷底调整两个耦合参数。第 70 轮时参数约为 $(0.9498, 1.7663)^\top$ ，损失约为 0.1620，并继续向黑色五角星表示的最小二乘解靠近。

这个实验表明，正规方程和梯度下降是在求解同一个最小二乘目标。正规方程直接计算终点，梯度下降则展示了参数怎样到达终点。后者更具一般性：当模型和损失变得复杂、解析解不再存在时，只要能够计算梯度，仍可以沿用相同的迭代优化思想。